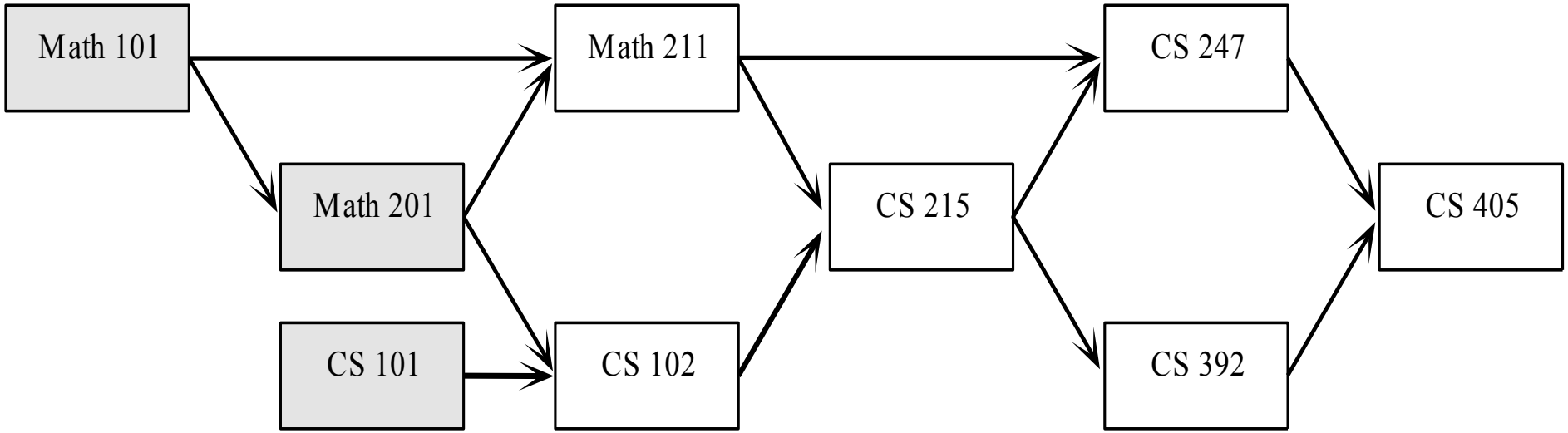
Table 12-1

Course	Prerequisite(s)	Barry has completed?
Math 101	None	Yes
Math 201	Math 101	Yes
CS 101	None	Yes
CS 102	CS 101; Math 201	No
CS 215	CS 102; Math 211	No
Math 211	Math 101;Math 201	No
CS 247	CS 215; Math 211	No
CS 392	CS 215	No
CS 405	CS 247; CS 392	No

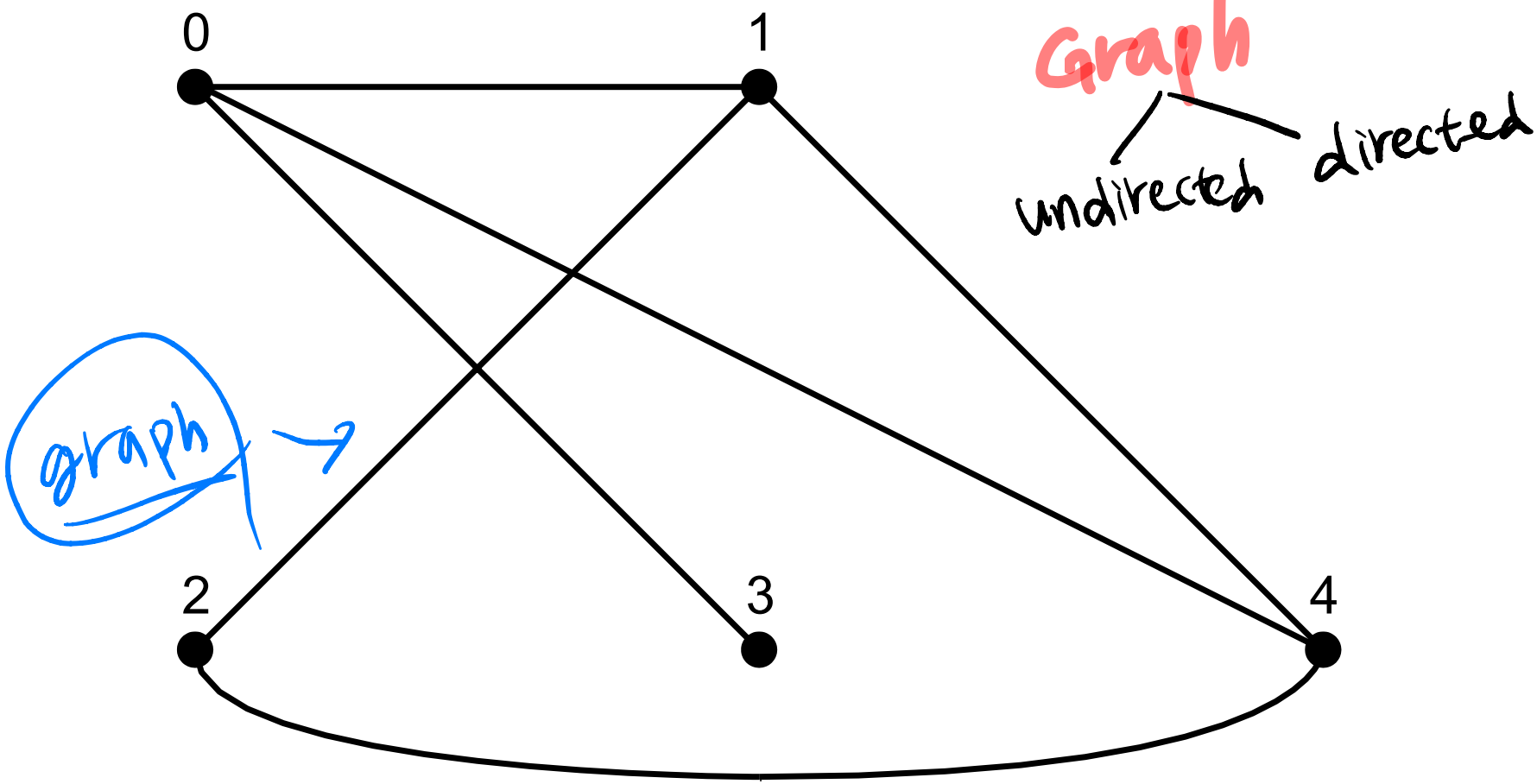
Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-1: Course Prerequisites in Graphical Form



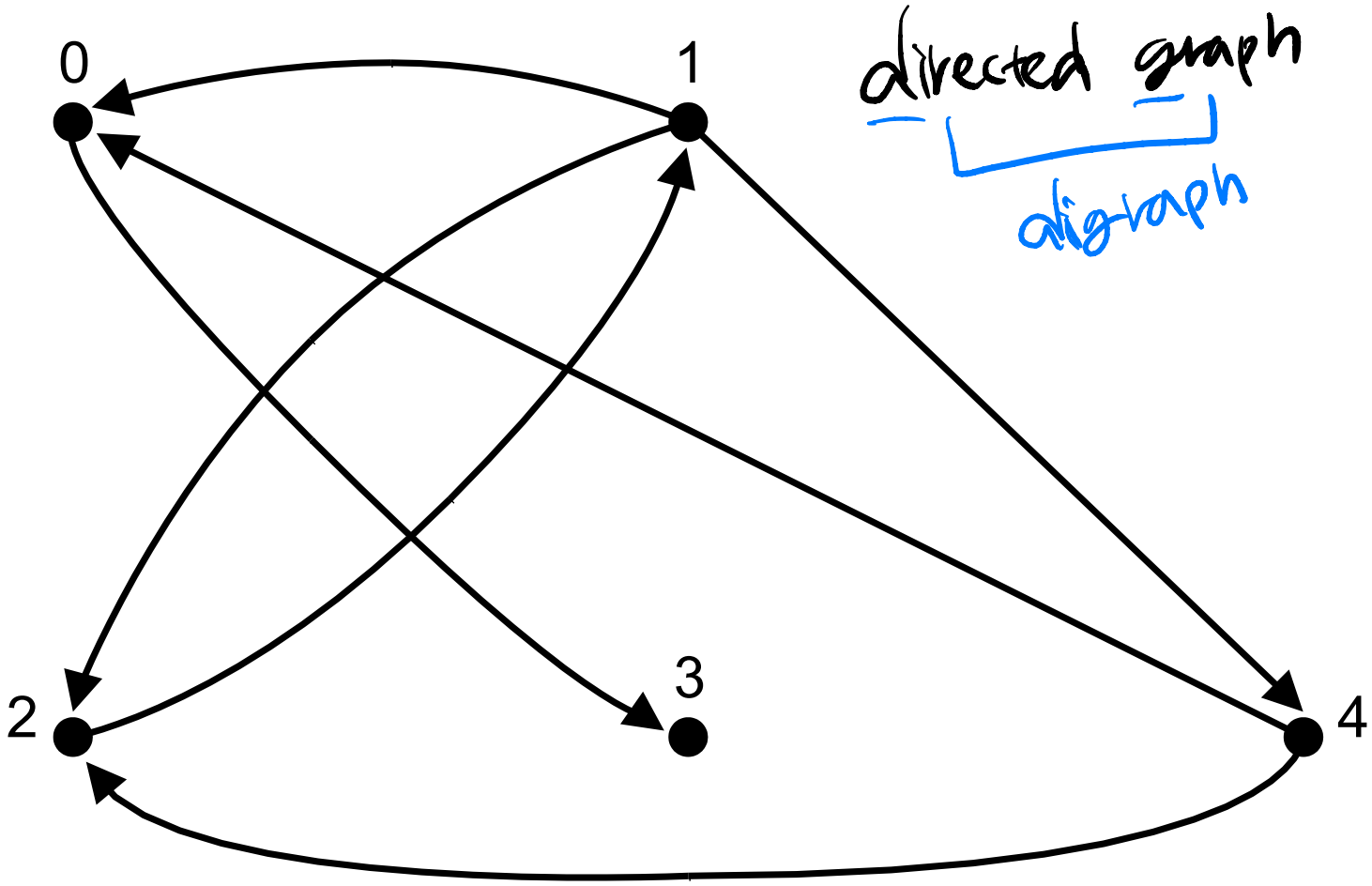
Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-2: An Example Graph



Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-3: An Example Digraph



directed graph

digraph

Copyright © 1997 Oxford University Press
All Rights Reserved

Basic concets

G= (V,E)

V={0,1,2,3,4} // node

E={{0,1},{0,3},{0,4},{1,2},{1,4},{2,4}} ← graph

Vertex 개수

n=|V|, m=|E| Edge 개수

E={(0,3),(1,0),(1,2),(1,4),(2,1),(4,0),(4,2)} ← digraph

|V|=1, |E|=0

n=5.

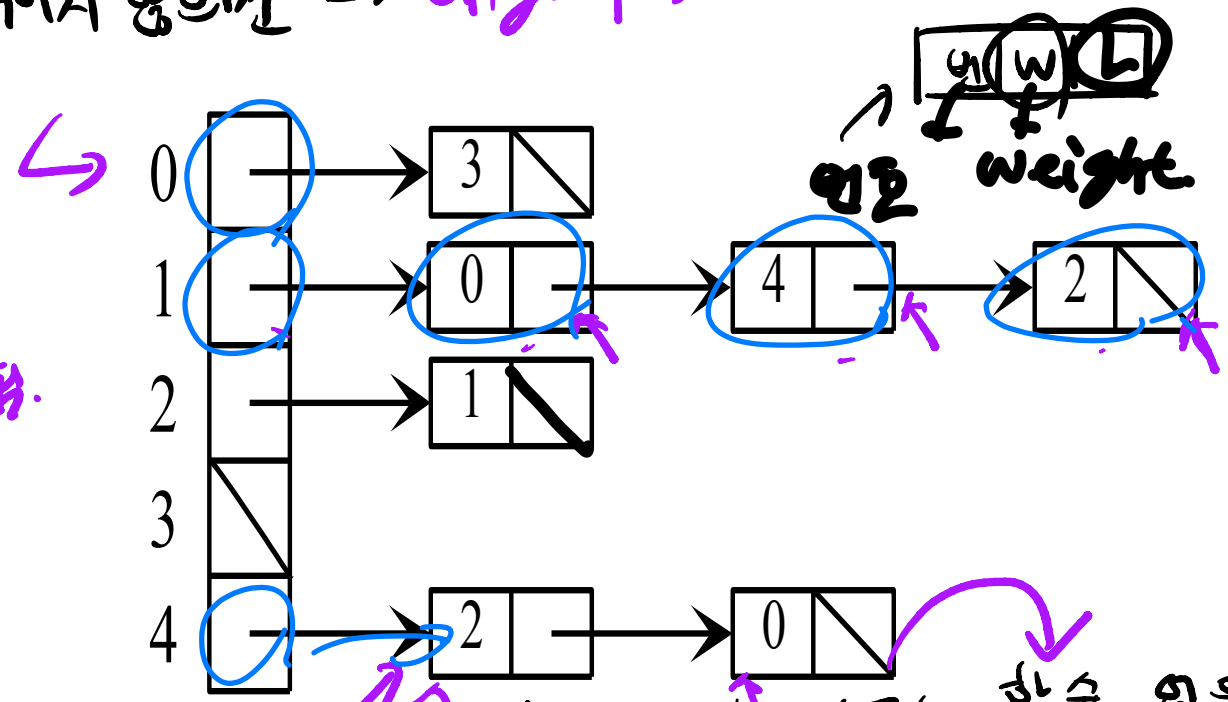
m=6

Copyright © 1997 Oxford University Press
All Rights Reserved

수정 그리는 문제

Figure 12-4: An Adjacency List of Figure 12-3

대칭적이지 않으면 → digraph



LA의 개수 → 2번 넣으면됨.

그러나 대칭적이라고 해서 graph 라고는 할 수 없음.

current

current.

Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-5: An Adjacency Matrix representation of Figure 12-3

도착

시작

	0	1	2	3	4
0	1	0	0	1	0
1	1	1	1	0	1
2	0	1	1	0	0
3	0	0	0	1	0
4	1	0	1	0	1

교차로

vertex(node)
: 역시 동일

많아 봤자

→ 4개, 8개 : 1

→ Ø이 많고 비효율

자기자신

에 대해 가는 길이

있다 → 1

없다 → 0

scan

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-1

AM : scan 해야 함. (비효율)

AL : traverse 하면 찾게 됨.

Characteristics

An Undirected Adjacency List Graph G = (V,E) stores an undirected graph so that vertex neighbors can be found efficiently.

The number of vertices in the graph, n = |V|, is fixed when the graph is created.

The vertices are labeled 0…n-1.

Operations:

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-1

int vertexSize()

Precondition: None.

Postcondtion: None.

Returns: The number of vertices in the graph, |V|.

int edgeSize()

Precondition: None.

Postcondtion: None.

Returns: The number of edges in the graph, |E|.

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-1

void addEdge(i,j)

Precondition: $0 \le i < n; 0 \le j < n; \{i, j\} \notin E.$

Postcondition: $\{i, j\} \in E$

int nextNeighbor(i)

Precondition: $0 \le i < n$

Postcondition: If the current iterator position is within the neighbor list, it's advanced to the next neighbor; if it's at the end of the neighbor list, it's reset to the beginning.

Returns: The next neighbor of i, or n if at the end of the list.

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-2

Characteristics

A <u>Directed Adjacency List Graph</u> G = (V,E) stores an directed graph so that vertex neighbors can be found efficiently. The number of vertices in the graph, n = |V|, is fixed when the graph is created. The vertices are labeled 0…n-1.

Operations:

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-2

int vertexSize()

Precondition: None.

Postcondition: None.

Returns: The number of vertices in the graph, |V|.

int edgeSize()

Precondition: None.

Postcondition: None.

Returns: The number of edges in the graph, |E|.

Copyright © 1997 Oxford University Press
All Rights Reserved

iterator
: OPP에서 배열, 2와 유사한 자료구조의 내부의 요소를 traversing 하는 객체.

ADT 12-2

void addEdge(i,j)

Precondition: $0 \le i < n; 0 \le j < n; (i, j) \notin E.$

Postcondition: $(i, j) \in E$ ← UALG와의 유연한 차이

int nextNeighbor(i)

Precondition: $0 \le i < n$ ← vertax 개수.

Postcondition: If the current iterator position is within the neighbor list, it's advanced to the next neighbor; if it's at the end of the neighbor list, it's reset to the beginning. : 처음으로 리셋됨, 돌아가서

Returns: The next neighbor of i, or n if at the end of the list.

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-3

{i,j} 연결이 잘 되었는지 파악

Characteristics

AL보다 효율적. ←

An Undirected Adjacency Matrix Graph G = (V,E) stores an undirected graph so that vertex connectivity can be queried efficiently. The number of vertices in the graph, n =|V|, is fixed when the graph is created. The vertices are labeled 0…n-1.

Operations:

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-3

int vertexSize()

Precondition: None.

Postcondition: None.

Returns: The number of vertices in the graph, |V|

int edgeSize()

Precondition: None.

Postcondition: None.

Returns: The number of edges in the graph, |E|

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-3

void addEdge(i,j)

Precondition: $0 \le i < n; 0 \le j < n; \{i, j\} \notin E.$

Postcondition: $\{i, j\} \in E$

bool edgeConnected(i,j)

Precondition: $0 \le i < n; 0 \le j < n.$

Postcondition: None.

Returns: True if and only if $\{i, j\} \in E$. Note that by convention $\{i, i\} \in E$.

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-4

Characteristics

A <u>Directed Adjacency Matrix Graph</u> G = (V,E) stores an directed graph so that vertex connectivity can be queried efficiently. The number of vertices in the graph, n = |V|, is fixed when the graph is created. The vertices are labeled 0…n-1.

Operations:

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-4

int vertexSize()

Precondition: None.

Postcondition: None.

Returns: The number of vertices in the graph, |V|.

int edgeSize()

Precondition: None.

Postcondition: None.

Returns: The number of edges in the graph, |E|.

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-4

void addEdge(i,j)

Precondition: $0 \le i < n; 0 \le j < n; (i, j) \notin E.$

Postcondition: $(i, j) \in E$

bool edgeConnected(i,j)

Precondition: $0 \le i < n; 0 \le j < n.$

Postcondition: None.

Returns: True if and only if $(i, j) \in E$. Note that by convention $(i, i) \in E$.

Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-10: Inheritance Hierarchy for Graphs

공통분모

① 부모 class 객체화? X

② 공통.

class → 상속 전용
객체 생성.

Graph. →

함수: vertexsize
edgesize.

변수: n,
m

덕이: addedge → 각 클래스에서 다른 내용을 포함함.

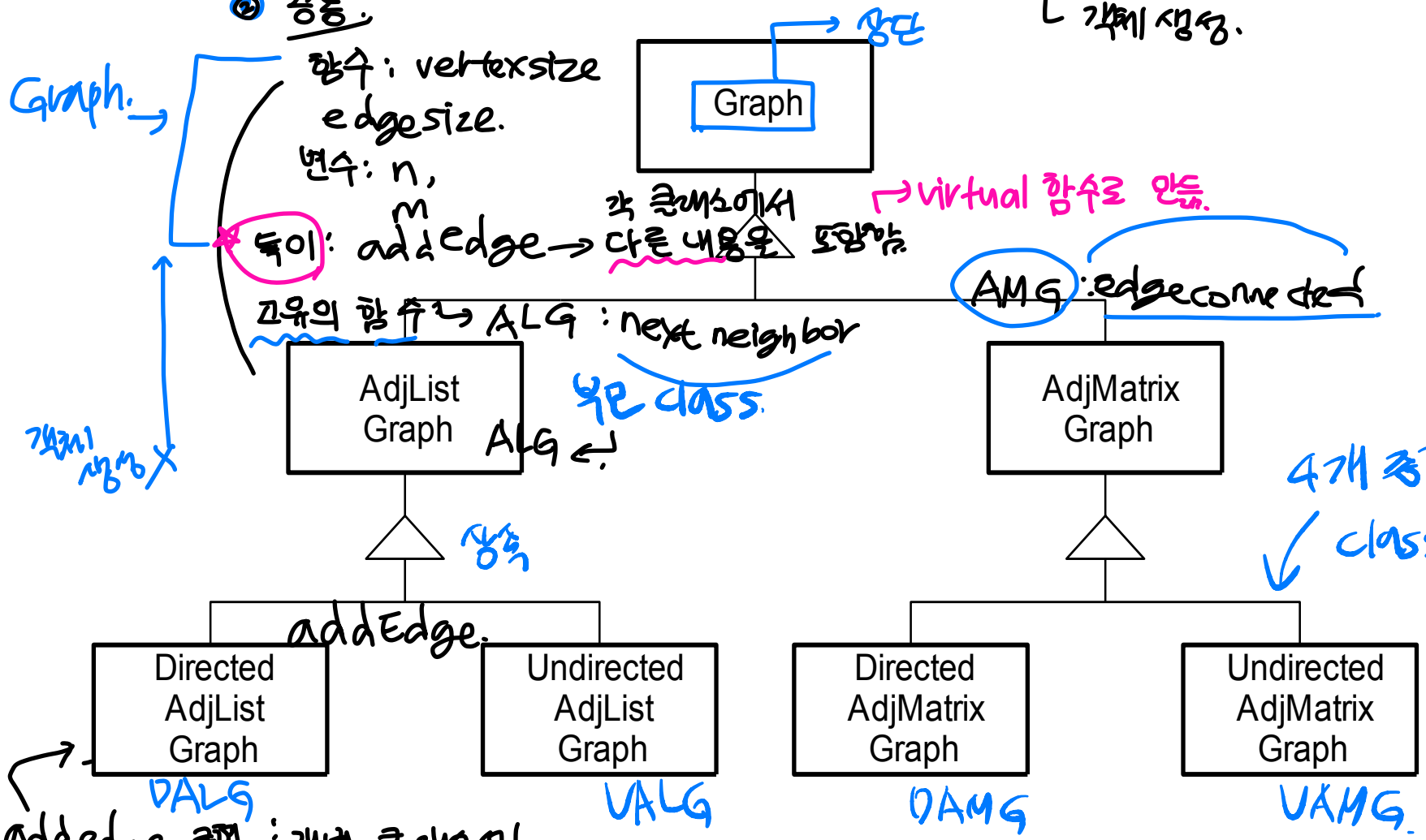
→ virtual 함수로 만듦.

고유의 함수 → ALG : next neighbor

AMG : edgeconnected

부모 class.

객체 생성 X



4개 종류 class.

addedge 구현. : 개별 클래스에 맞게.

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-1

virtual void addEdge
(int fromV, int toV)=0.

```
// cx12-1.h
// Code Example 12-1: Header for graph hierarchy
#ifndef __MB_CX12_1__
#define __MB_CX12_1__

class graph {
public:
  graph(int size) : n(size), m(0) { }
  virtual int vertexSize() { return n; }
  virtual int edgeSize() { return m; }
  virtual void addEdge(int fromV, int toV) = 0; // specify abstract class
protected:
  int n; // number of vertices
  int m; // number of edges
};
#endif
```

virtual 인 편은 x.

안에서 정의 ⇒ inline 함수.

객체 만들 수 x.

graph graph; x

← 자식 class에서 사용

: pure virtual
→ class 정의 안해도 됨

: 하나 이상의 멤버 함수를 순수 가상 함수로 선언한 클래스

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-5: List ADT

Characteristics:

A List L stores items of some type, called ListElementType. The items in the List are ordered; the Lists (1,2) and (2,1) are distinct.

Operations:

void L.insert(ListElementType elem)

Precondition: None.

Postcondition: L = L with an instance of elem added to the end of L.

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-5

bool L.first(ListElementType &elem)

Precondition: None.

Postcondition: If the list is empty, none. Otherwise, the variable elem contains the first item in L; the "next" item to be returned is the second in L.

Returns: **true** if and only if there is at least one element in L.

Copyright © 1997 Oxford University Press
All Rights Reserved

ADT 12-5

bool L.next(ListElementType &elem)

Precondition: The “first” operation has been called at least once.

Postcondition: Variable elem contains the next item in L, if there is one, and the next counter advances by one; if there is no next element, none.

Returns: **true** if and only if there was a next item.

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-2

```
// cx12-2.h
// Code Example 12-2: Header, List class with List Iterator class

#ifndef __MB_CX12_2__
#define __MB_CX12_2__

#include "dslib.h"

template < class ListElementType >
class ListIter;
```

선언 → 뒤에서 정의할 경우
앞에서 해야 함.

Copyright © 1997 Oxford University Press
All Rights Reserved

생성자 초기화 클래스명( ) : 멤버변수명(값 또는 변수) { 추가 초기화 }

Code Example 12-2

```
template < class ListElementType >
class List {
public:
   List() : head(0) { }
   virtual void insert(const ListElementType & elem);
   friend class ListIter < ListElementType >;
protected:
   struct Node;
   typedef Node * Link;
   struct Node {
      ListElementType elem;
      Link next;
   };
```

friend class ListIter < ListElementType >;

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-2

```
  Link head;
};

template < class ListElementType >
class ListIter {
public:
  ListIter(const List < ListElementType > & l, ListElementType endFlag)
    : myList(l), myEndFlag(endFlag), iterPtr(0) { }
  virtual ListElementType operator++();
protected:
  const List < ListElementType > & myList;
  List < ListElementType >::Link iterPtr;
  ListElementType myEndFlag;
};
#endif
```

순회하는 자.

원본 클래스의 객체 받아서
이름을 가지고 있음

연산자 오버로딩.
→ nextneighbor( ) 을 대체.

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-3

```
// cx12-3.cpp
// Code Example 12-3: Implementation, List class with List Iterator class

#include "cx12-2.h"

template < class ListElementType >
ListElementType
ListIter < ListElementType > :: operator++()
{
  if (iterPtr == 0)
    iterPtr = myList.head;
  else
    iterPtr = iterPtr->next;
  if (iterPtr)
    return iterPtr->elem;
  else
```

첫번째 노드

Locking

존재하지 않을 수 있다.

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-3

```
    return myEndFlag;
}

template < class ListElementType >
void
List < ListElementType > :: insert(const ListElementType & elem)
{
  Link addedNode = new Node;
  assert(addedNode);
  addedNode->elem = elem;
  addedNode->next = head;
  head = addedNode;
}
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-4

Adjacency List.

AL graph.

```
// cx12-4.h
// Code Example 12-4: Header for adjacency list base class

#ifndef __MB_CX12_4__
#define __MB_CX12_4__

#include "cx12-1.h" // graph base class
#include "cx12-2.h" // list class

typedef List < int > IntList;
typedef ListIter < int > IntListIter;
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-4

```
class ALGraph : public graph {
public:
   ALGraph(int size) : graph(size)
      { vertexList = new IntList[n]; assert(vertexList); }
   friend class NeighborIter;
protected:
   IntList * vertexList;
};

class NeighborIter : public IntListIter {
public:
   NeighborIter(const ALGraph & G, int startVertex) :
   IntListIter (G.vertexList[startVertex], G.n)
      { assert(startVertex < G.n); }
};
#endif
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-5

```
// cx12-5.h
// Code Example 12-5: Header for Undirected Adjacency List Graphs

#ifndef __MB_CX12_5__
#define __MB_CX12_5__

#include "cx12-4.h"
class UALGraph : public ALGraph {
public:
  UALGraph(int size) : ALGraph(size) { }
  virtual void addEdge(int fromV, int toV);
};
#endif
```

UALG→ addEdge 만 추가.

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-6

```
// cx12-6.cpp
// Code Example 12-6: Implementation file for Undirected Adjacency List Graph

#include "cx12-5.h"

void
UALGraph::addEdge(int fromV, int toV)
{
  assert(fromV < n && fromV >= 0 && toV < n && toV >= 0);
  vertexList[fromV].insert(toV);
  vertexList[toV].insert(fromV);
  m++;
}
```

방향성X

상방

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-7

```
// cx12-7.h
// Code Example 12-7: Header for Directed Adjacency List Graph

#ifndef __MB_CX12_7__
#define __MB_CX12_7__

#include "cx12-4.h"  // ALGraph -- Adjacency List Base Class

class DALGraph : public ALGraph {
public:
   DALGraph(int size) : ALGraph(size) { }
   virtual void addEdge(int fromV, int toV);
};
#endif
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-8

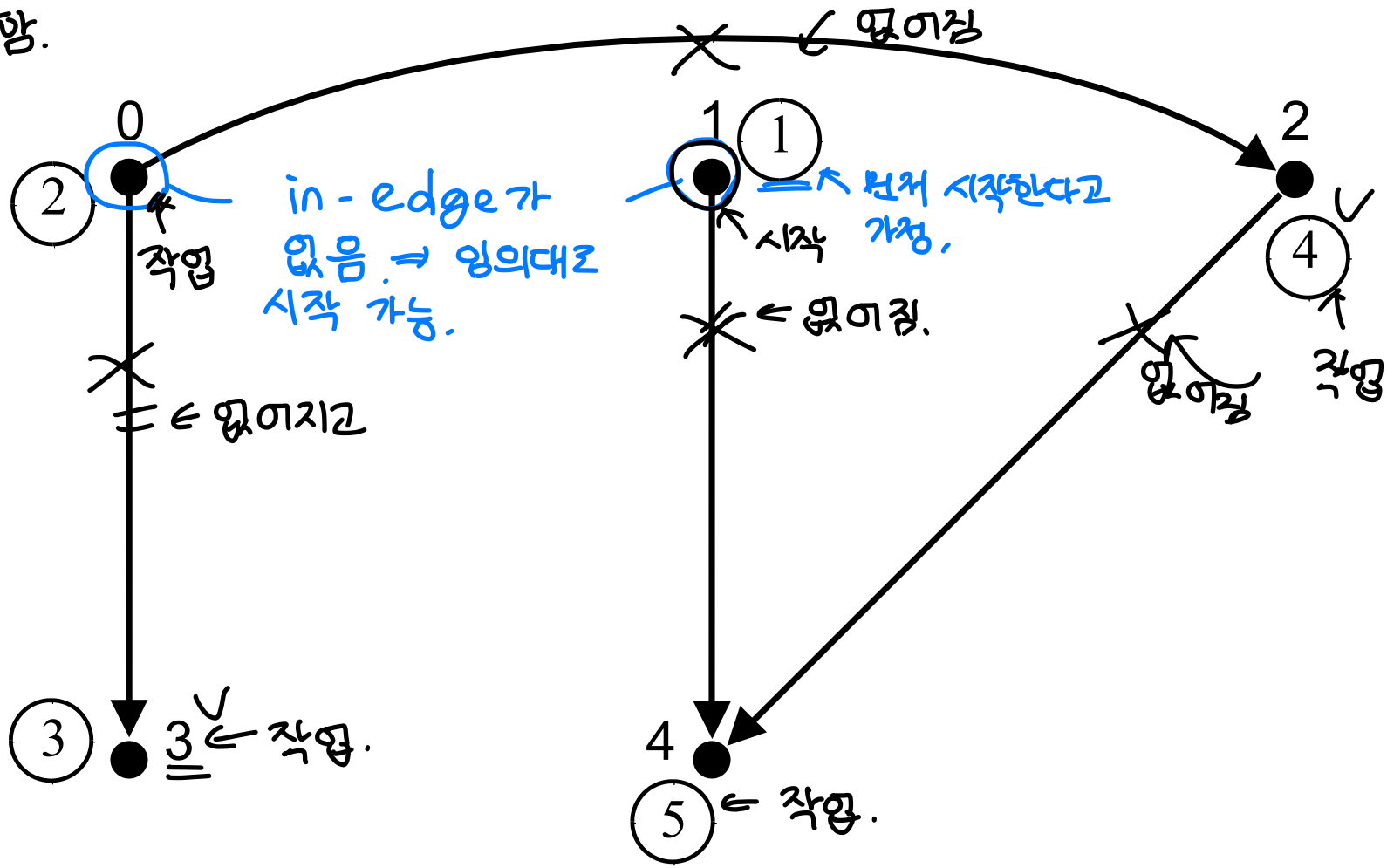
```
// cx12-8.cpp
// Code Example 12-8: Implementation file for Directed Adjacency List Graph

#include "cx12-7.h"

void
DALGraph::addEdge(int fromV, int toV)
{
  assert(fromV < n && fromV >= 0 && toV < n && toV >= 0);
  vertexList[fromV].insert(toV);
  m++;
}
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Topological sorting → data flow

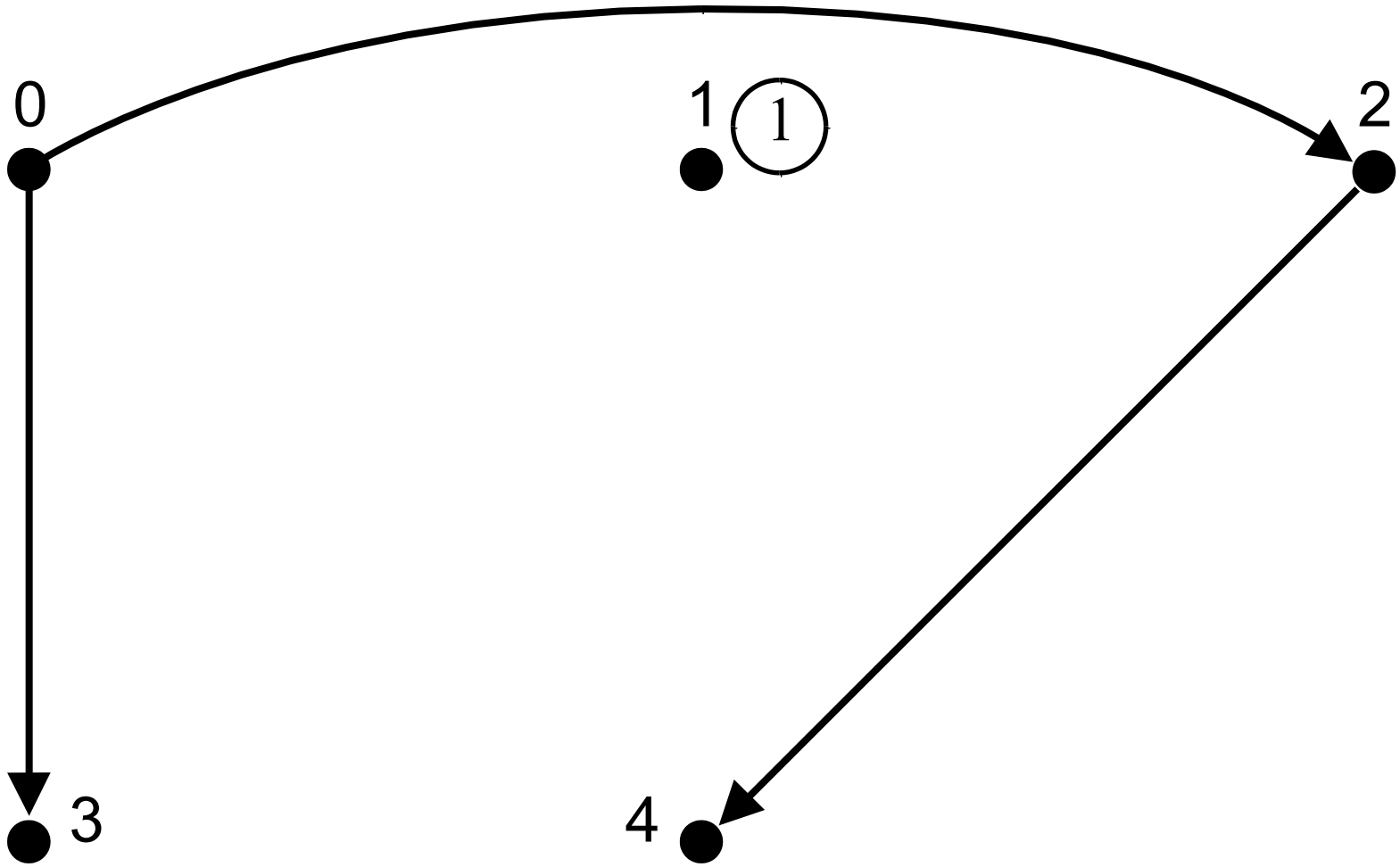
→ 자기에게 들어오는 edge가 없으면 시작할 수 있고, in-edge가 있으면 시작 못함.

Partial order example



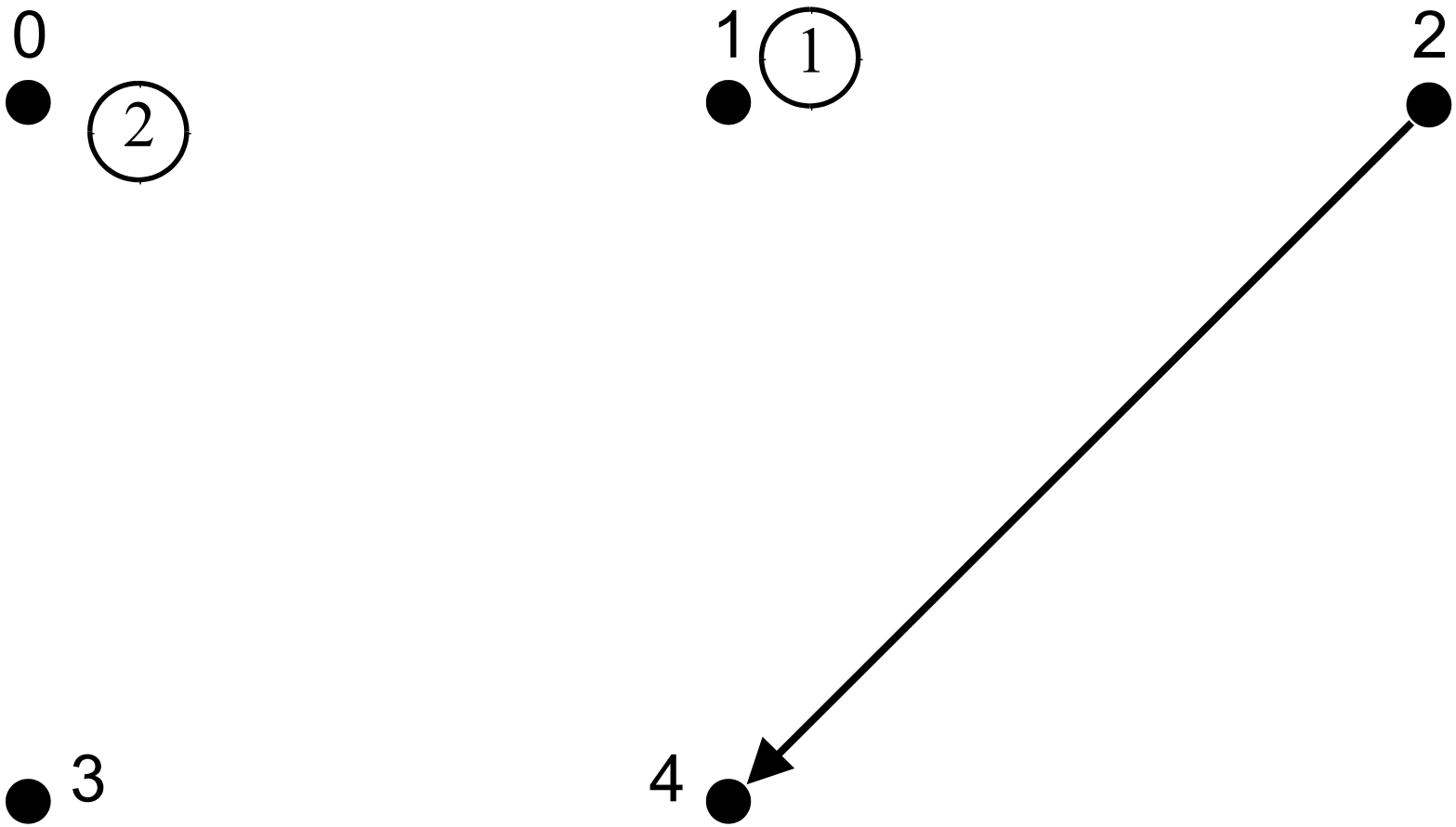
Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-12: Topsort algorithm <u>after one step</u>



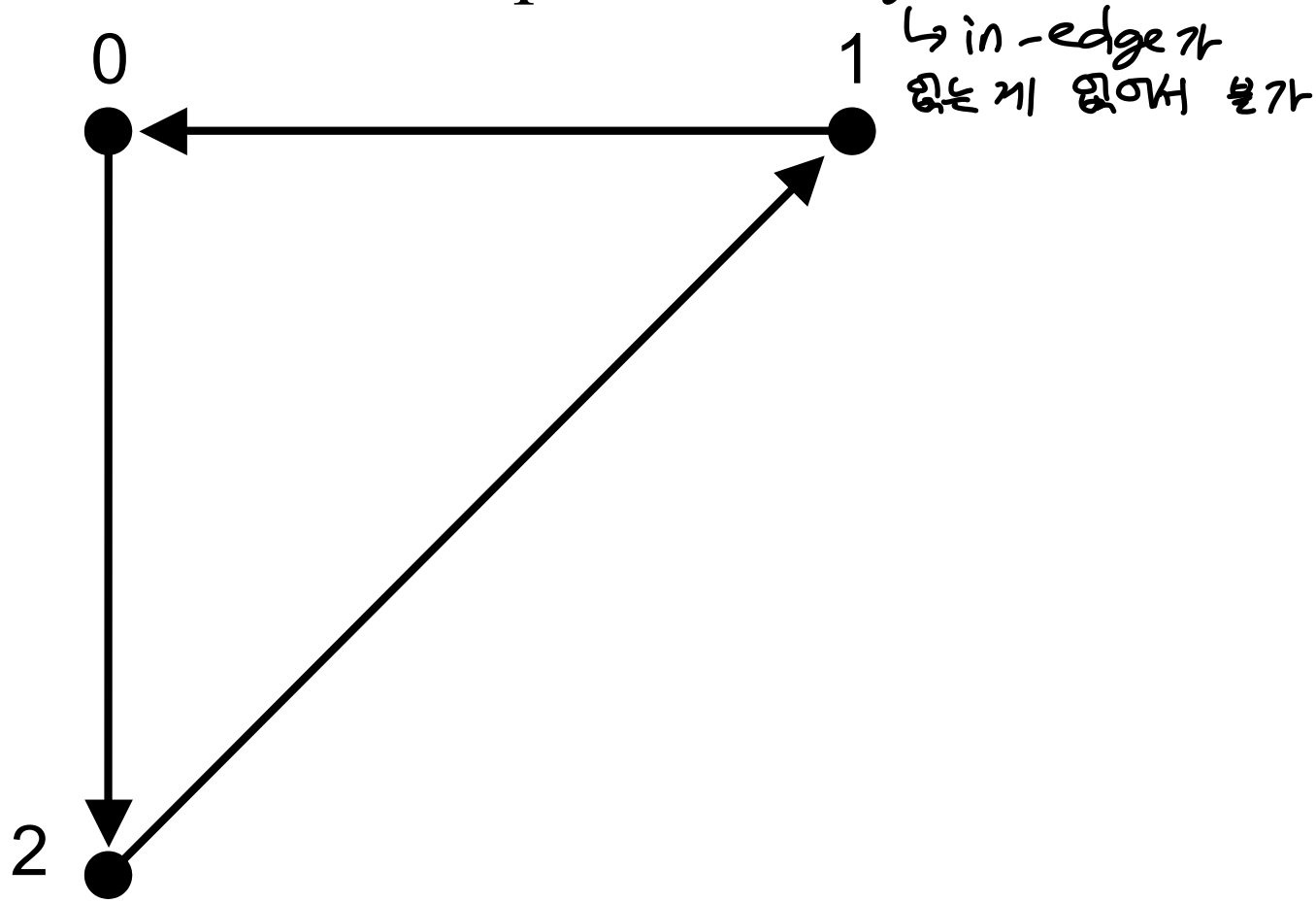
Copyright © 1997 Oxford University Press
All Rights Reserved

12-13: Topsort after the second step



Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-14: Graph with a cycle



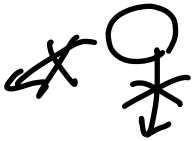
Copyright © 1997 Oxford University Press
All Rights Reserved

Algorithm 12-1:TOPSORT

nextLabel = 1 :순서

find all vertex with no in-edges and push them onto a stack LIFO

while the stack is not empty do



pop a vertex v from the stack

label(v) = nextLabel : 순서 부여

add 1 to nextLabel 순서올림

vertex에서 나가는 outedge 제거

remove all other edges from v

if any neighbor of v now has no in-edge, push it onto the stack

if all vertices labeled,

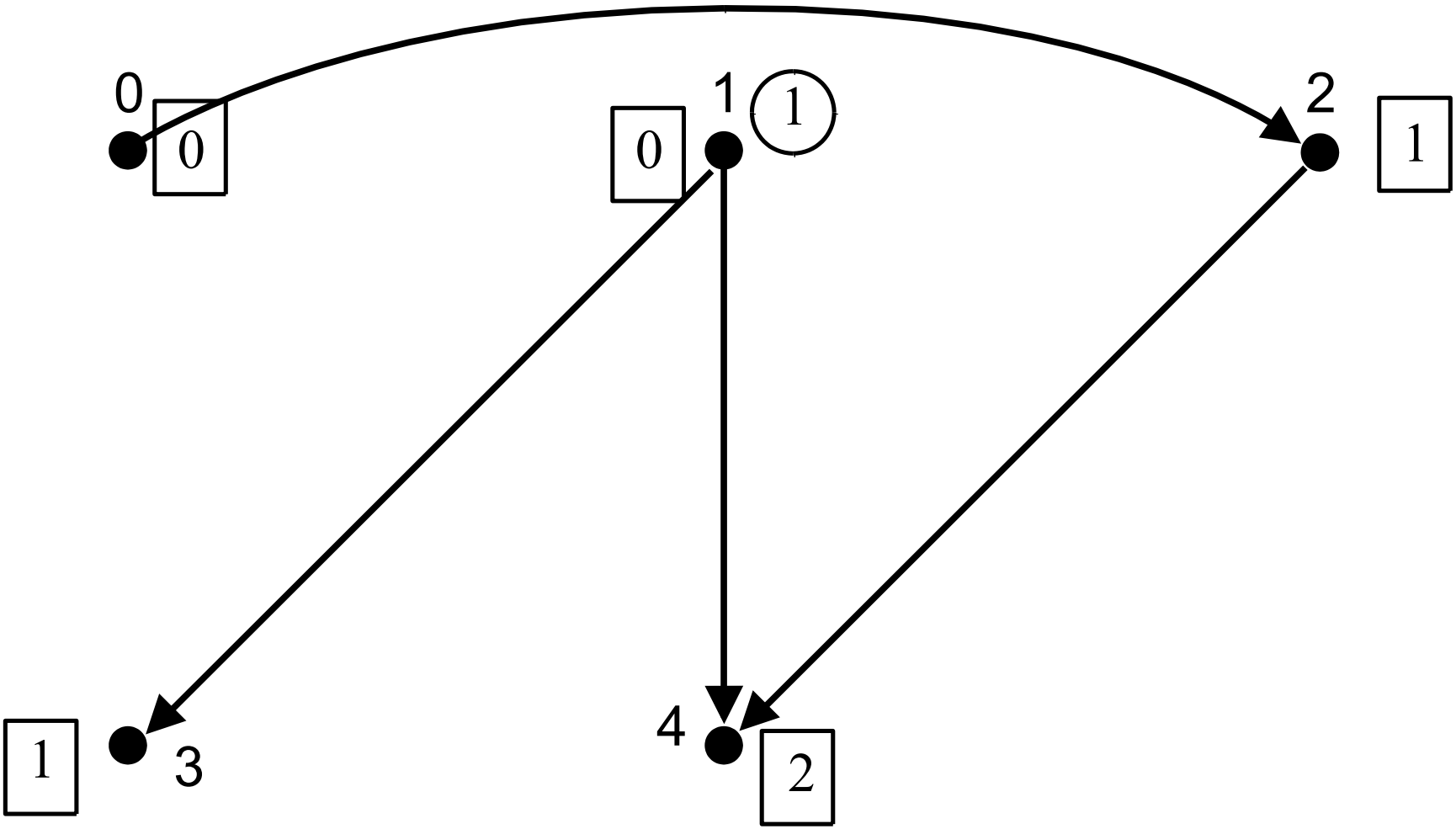
report the labels

else

report that the digraph has a cycle

Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-15: One Step in a Topological Sort



Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-9

```
// cx12-9.cpp
// Code Example 12-9: Topological Sort

#include "dslib.h"
#include "cx12-7.h"  // directed adjacency list graphs
#include "sx8-1.h"   // stacks (supplemental version)
#include "cx9-4.h"   // queues
#include <fstream.h>

int main()
{
  // read graph from a file
  // first entry is size of graph
  const char * inFileName = "graph.dat";
  ifstream ifs(inFileName);
  assert(ifs); // make sure graph exists
```

graph를 읽어옴.

Directed

DALG.

graph 구조

text 파일.

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-9

```
int n;
ifs >> n;
DALGraph G(n);
cout << "Created graph; n = " << G.vertexSize() << endl;
// now read in the edges and add to the graph
int u, v;
while ( ifs >> u ) {
   ifs >> v;
   G.addEdge(u,v);
}
cout << "Edges in graph: m = " << G.edgeSize() << endl;
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-9

```
// count the number of in-edges for each vertex
  int * vertices(new int[n]);
  assert(vertices);
  for (u = 0; u < n; u++)
    vertices[u] = 0;
  for (u = 0; u < n; u++) {
    NeighborIter ni(G,u);
    while ((v = ++ni) != n)
      vertices[v]++;
  }
```

⇒ # of inEdge

in-edge 초기화시킴

u v

(v = ++ni) != n

v 야 inEdge 1개 있어!

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-9

```
// put vertices with no in-edge onto a stack
  Stack < int > s;
  for (u = 0; u < n; u++)
     if (vertices[u] == 0)
        s.push(u);
  if (s.isEmpty()) {
     cout << "graph has a cycle!\n";
     return 0;
  }
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-9

Stack s에 in-edge가 없는 것들은 넣었음.

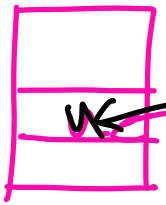
```
// begin topological sort
  // As each vertex is identified, put it into a queue and
  // decrement the number of in-edges for its neighbors
  int count = 0; // number of vertices found so far
  Queue < int > sortedEdges;
  while (!s.isEmpty()) {
    count++;
    u = s.pop();
    sortedEdges.enqueue(u);
    // reduce in count for u's neighbors;
    // for each that goes to zero, put on stack
    NeighborIter ni(G,u);
```

← 순번을 알려줌.

u = s.pop()

sorting 된 순서가 + Queue에 들어가는 순서

키번



(checking) 과정

u 이웃

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-9

```
while ((v = ++ni) != n) {
    --vertices[v];
    if (vertices[v] == 0)
        s.push(v);
}
```

→ Inedge 줄임

← 독립

u → v

u → v

(v=++ni) != n)

vertices[v]--

vertices[v]==0

s.push(v)

```
// check results
if (count < n)
    cout << "Couldn't complete top sort -- cycle present.\n";
cout << "Ordering for top sort: \n";
while (!sortedEdges.isEmpty())
    cout << sortedEdges.dequeue() << '\t';
cout << endl;
return n;
}
```


Copyright © 1997 Oxford University Press
All Rights Reserved

실습예제

1. TopSort라는 프로젝트를 생성한다.
2. 이전에 사용한 stack 과 queue template 파일들을 복사해서 프로젝트에 추가한다
3. graph.h 새항목추가에서 20쪽의내용 넣음
4. List.h 을 새항목추가해서 24-28쪽 내용을 추가
   1. List class 내용을 먼저 다 적고, 그다음에 ListIter class 내용을 적는다
   2. typename List < ListElementType >::Link iterPtr; 이렇게 수정
5. ALGraph.h 를 새항목추가해서 29-30쪽 내용을 추가한다. Include 파일은은 graph.h, list.h 를 포함함.
6. DALGraph 클래스를 추가. 33-34 내용을 .h 와 .cpp 파일에 추가한다
7. Main 함수에 41-46쪽 내용을 복사한고 필요한 header 파일 추가한다
8. 작업디렉토리에 graph.dat 파일을 만들고 내용은 35쪽 그림을 기준으로 만든다

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-10

```
// cx12-10.h
// Code Example 12-10: Header for adjacency matrix base class

#ifndef __MB_CX12_10__
#define __MB_CX12_10__

#include "dslib.h"
#include "cx12-1.h" // graph base class

class amGraph : public graph {
public:
   amGraph(int size);
   virtual bool edgeMember(int fromV, int toV);
protected:
   int * * am; // am points to pointers to int
};
#endif
```

AM graph

DAMG UAMG

addedge.

edge connected

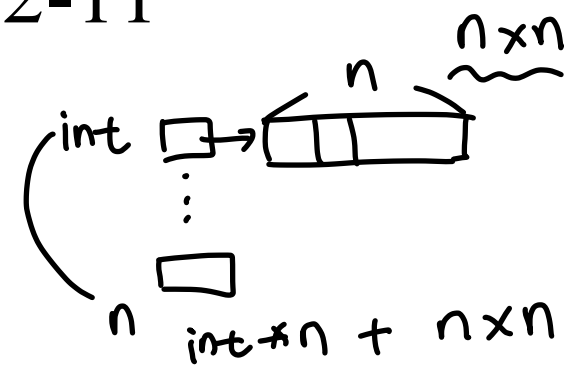
Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-11

```
// cx12-11.cpp
// Code Example 12-11: Implementation for adjacency matrix base class

#include "cx12-10.h"
amGraph::amGraph(int size) : graph(size)
{
  int i;
  am = new int * [n]; // make an array of pointers to int
  assert(am);
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-11




```
for (i = 0; i < n; i++) {
      am[i] = new int[n];  // make an array of ints
      assert(am[i]);
      int j;
      for (j = 0; j < n; j++) // initialize the array to 0
         am[i][j] = 0;
   }
}

bool amGraph::edgeMember(int fromV, int toV)
{
   assert (fromV < n && toV < n && fromV >= 0 && toV >= 0);
   return bool(am[fromV][toV] != 0);
}
```

연결이 된 상태 : 참
안된 〃 : Ø

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-12

```
// cx12-12.h
// Code Example 12-12: Header for directed adjacency matrix class

#ifndef __MB_CX12_12__
#define __MB_CX12_12__

#include "cx12-10.h" // adjacency matrix base class

class dAMGraph : public amGraph {
public:
  dAMGraph(int size, int initialValue = 0) :
      amGraph(size) { }
  virtual void addEdge(int fromV, int toV);
};

#endif
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-13

```
// cx12-13.cpp
// Code Example 12-13: Implementation for directed adjacency matrix class

#include "cx12-12.h"
void dAMGraph::addEdge(int fromV, int toV)
{
  assert(fromV < n && toV < n && fromV >= 0 && toV >= 0);
  if (!edgeMember(fromV, toV)) {
    m++;
    am[fromV][toV] = 1;
  }
}
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-14

```
// cx12-14.h
// Code Example 12-14: Header for undirected adjacency matrix class

#ifndef __MB_CX12_14__
#define __MB_CX12_14__

#include "cx12-10.h"

class uAMGraph : public amGraph {
public:
  uAMGraph(int size, int initialValue = 0) :
      amGraph(size) { }
  virtual void addEdge (int fromV, int toV);
};
#endif
```

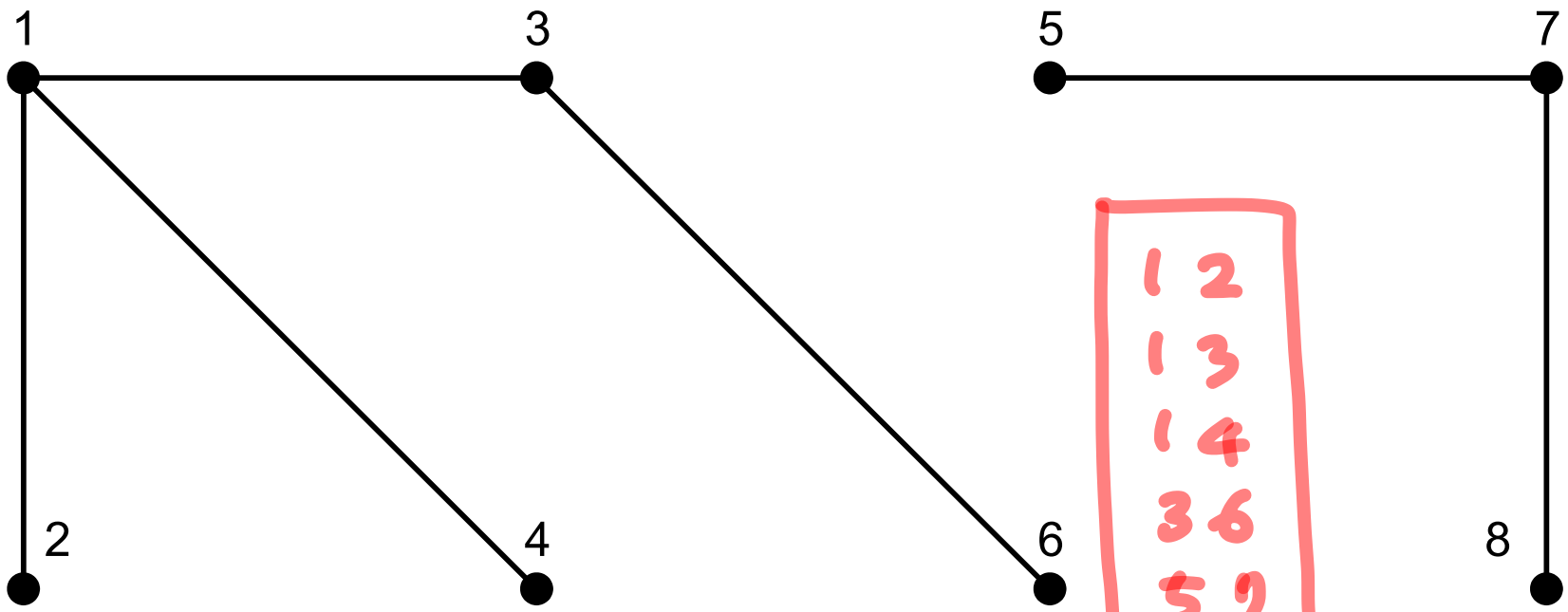

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-15

```
// cx12-15.cpp
// Code Example 12-15: Implementation for undirected adjacency matrix class

#include "cx12-14.h"

void uAMGraph::addEdge(int fromV, int toV)
{
  assert(fromV < n && toV < n && fromV >= 0 && toV >= 0);
  if (!edgeMember(fromV, toV)) {
    m++;
    am[fromV][toV] = 1;
    am[toV][fromV] = 1;
  }
}
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-16: A Communications Network



1 2
1 3
1 4
3 6
5 7
7 8

2 – 1 – 4
간접적으로 연결.

2 – 1 – 3.
2 – 1 – 3 – 6] transitive closure

Copyright © 1997 Oxford University Press
All Rights Reserved

Definitions

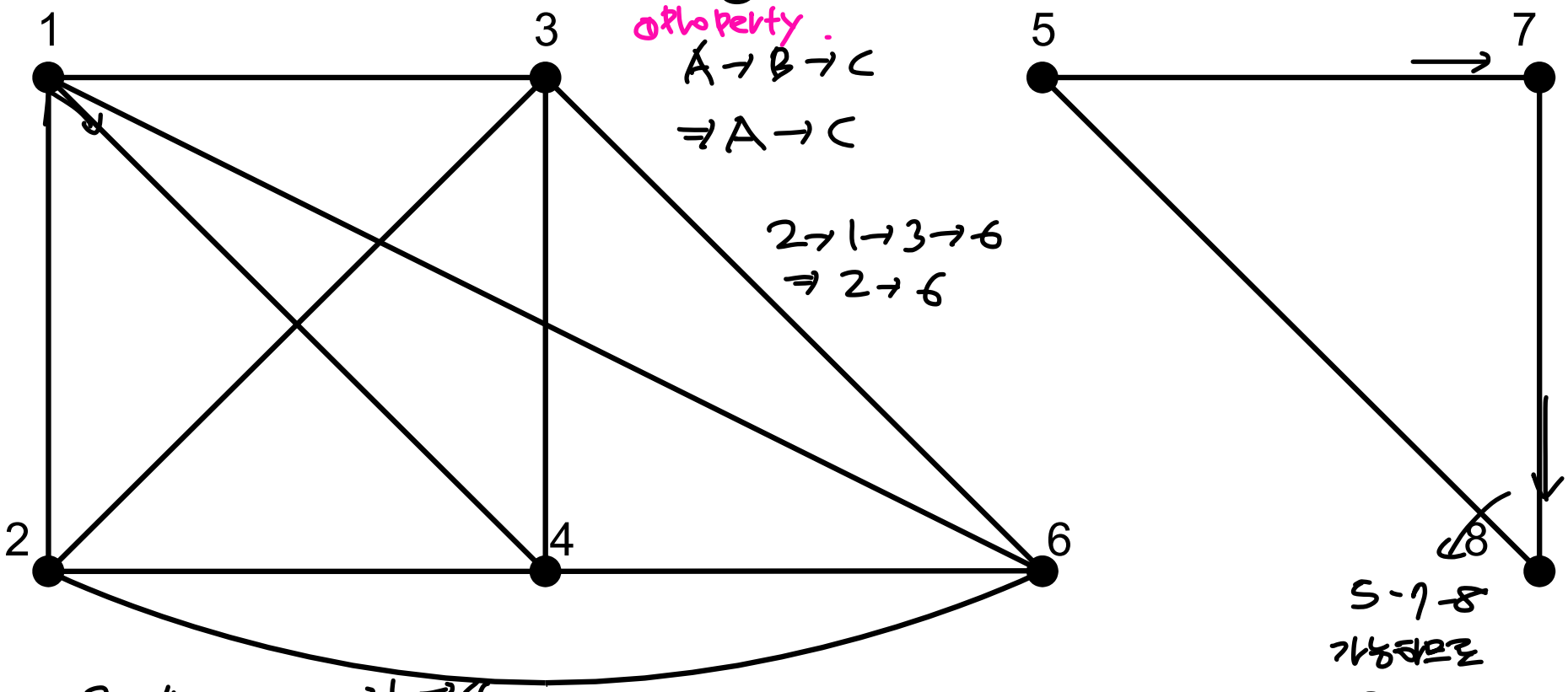
12-1 A sequence $[v_0, v_1, \ldots, v_k]$ is a path from v_0 to v_k in an undirected graph $G = (V,E)$ if and only if $\{v_i, v_{j+1}\} \in E$ for all $1 \le i < k$. The length of the path is k.

중복X

12-2 For a graph $G = (V, E)$, the transitive closure graph $G^* = (V, E^*)$ has edge $\{v_1, v_2\} \in E^*$ if and only if there's a path $[v_1, \ldots, v_2]$ in graph G.

Copyright © 1997 Oxford University Press
All Rights Reserved

Figure 12-17: ~~Transitive Closure~~ of Graph from Figure 12-16



property.

A→B→C

⇒A→C

2→1→3→6

⇒2→6

2→4 : = 2→1→4

5→7→8

가능하므로

5→8

transitive closure.

Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-16

```
// cx12-16.cpp
// Code Example 12-16: Transitive Closure

#include "dslib.h"
#include "cx12-14.h"
#include <fstream.h>

int main()
{
   const char * inFileName = "graph2.dat";
   // read graph from a file
   // first entry is size of graph
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-16

```
ifstream ifs(inFileName);
assert(ifs);
int n;
ifs >> n;
uAMGraph G(n);
cout << "Created graph; n = " << G.vertexSize() << endl;
int u, v;
while ( ifs >> u ) {
   ifs >> v;
   G.addEdge(u,v);
}
```


Copyright © 1997 Oxford University Press
All Rights Reserved

Code Example 12-16

```
cout << "Edges in graph: m = " << G.edgeSize() << endl;
  int step;
  for (step=0; step < n; step++)  : 각 단계를 거쳐감
    for (u = 0; u < n; u++)
      for (v = 0; v < n; v++)
        if (G.edgeMember(u,step) && G.edgeMember(step,v))
          G.addEdge(u,v);
  // print results
  for (u=0; u < n; u++) {
    cout << u << "\t: ";
    for (v = 0; v < n; v++)
      cout << (G.edgeMember(u,v)? "T " : "F ");
    cout << endl;
  }
  return 0;
}
```

complexity : $\Theta(n^3)$

Copyright © 1997 Oxford University Press
All Rights Reserved

실습예제

1. TransitiveClosure 라는 프로젝트를 만든다
2. TopSort 프로젝트의 graph.h 파일을 복사해서 현재 프로젝트에 추가
3. amGraph 클래스를 추가하고 내용은 48에서 50 을 복사
4. UAMGraph 클래스를 추가하고 내용은 53-54
5. Main 함수에 58-60 쪽 내용을 복사
    1. Main 함수에서 addEdge 할때 u-1, v-1로 수정
6. 작업 디렉토리에 graph2.dat파일을 55쪽 그림에 맞게생성

Copyright © 1997 Oxford University Press
All Rights Reserved

test ch4 → 4문제

Red-black tree → 시험엔 x

Queue → 문제 배점 ↓

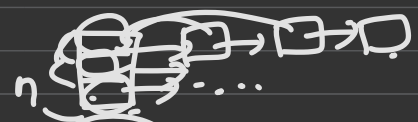
실습예제 중 한 개 나옴.

3번↑ 해보기

성격 확인 및 문의는 이메일로

Top Sort

```
int main()
{   int n;
    ifs >> n;
    DALGraph G(n);
    cout << " created graph; n =" << G.vertexSize() << endl;
    int u, v;
    while (ifs >> u) {
        ifs >> v;
        G.addEdge(u,v);
    }   int * vertices (new int(n));

    for (u=0; u<n; u++)
        vertices[u] = 0;
    for (u=0; u<n; u++) {
        NeighborIter. ni (G,u);
        while((v=++ni) != n)
            vertices[v]++;
```



number of edge.

```
Stack <int> s;
for (u=0; u<n; u++)
    if (vertices[u]==0)
        s.push(u)
if (s.isEmpty()) {
    cout << graph has a cycle!";
    return 0;
```

v 연결

u 개수 안 더하고 집어넣음.

```
int count=0;
Queue<int> sortedEdges;
while (!s.isEmpty()){
    count++
    u=s.pop();
    sortedEdges.enqueue(u);
    Neighbor
```

```
NeighborIter ni(g,u);
                I
while ((v=*ni)!=0){   StartVertex
    --vertices[v]
    if(vertices[v]==0)
      s.push(v)
}
```